



ΛProof Capability Token Specification

Version 1.0.0

Abstract

This document specifies the format, structure, and signature verification procedures for ΛProof Capability Tokens. These tokens authorize specific actions within the ΛProof ecosystem by cryptographically binding an evaluated intent to a time-bounded, delegable authorization.^[1]

Token Philosophy

ΛProof tokens follow the **capability model** rather than access control lists (ACLs). A token holder possesses the authority to perform a specific action—no identity lookup or centralized permission check is required at execution time. This aligns with ΛProof's principle that **authorization follows intent**, not identity.

Key properties:

- **Self-contained:** All authorization information is in the token
 - **Cryptographically verifiable:** Any party can verify without contacting a central server
 - **Delegable:** Tokens can grant sub-capabilities to other agents
 - **Time-bounded:** Every token has explicit validity windows
 - **Intent-bound:** Each token is cryptographically tied to a specific evaluated intent
-

Token Structure

ΛProof tokens use a three-part structure similar to JWT, encoded as `header.payload.signature` with Base64URL encoding:^{[2][3]}

ΛProof Capability Token	
Header (Base64URL)	
<pre>{ "alg": "EdDSA", // Signature algorithm "typ": "LPCT", // ΛProof Capability Token "kid": "did:key:z6Mk..." // Key identifier (DID) }</pre>	
Payload (Base64URL)	
<pre>{ "iss": "did:key:z6Mk...", // Issuer DID "sub": "did:key:z6Mk...", // Subject (who can use this) "aud": "adapter:email", // Target adapter/resource "iat": 1736365200, // Issued at (Unix timestamp) "nbf": 1736365200, // Not before "exp": 1736368800, // Expires at "nonce": "a1b2c3d4...", // Unique nonce (replay prevention) "intent_hash": "sha256:...", // Bound intent hash "cap": { // Capability specification "action": "send", "resource": "mailto:*@example.com", "constraints": {...} }, "proof_level": "ATTESTATION", // Required proof level "obligations": [...], // Required evidence/conditions "delegation_depth": 0, // How many times delegated "prf": ["cid:bafy..."] // Proof chain (delegation refs) }</pre>	
Signature (Base64URL)	

EdDSA signature over: Base64URL(header) + "." + Base64URL(payload)

Header Fields

Field	Type	Required	Description
alg	String	Yes	Signature algorithm. MUST be one of: EdDSA (Ed25519), ES256 (P-256), ES256K (secp256k1)
typ	String	Yes	Token type. MUST be LPCT (ΛProof Capability Token)
kid	String	Yes	Key identifier as DID. MUST use did:key method

Algorithm Preference: Ed25519 (EdDSA) is RECOMMENDED for new implementations due to performance and security properties.

Payload Fields

Identity Claims

Field	Type	Required	Description
iss	DID	Yes	Issuer—the principal who created and signed this token
sub	DID	Yes	Subject—the principal authorized to use this token

<code>aud</code>	String	Yes	Audience—the adapter or resource this token targets
------------------	--------	-----	---

Temporal Claims

Field	Type	Required	Description
<code>iat</code>	Integer	Yes	Issued-at timestamp (Unix seconds, UTC)
<code>nbf</code>	Integer	No	Not-before timestamp. If omitted, token is valid from <code>iat</code>
<code>exp</code>	Integer	Yes	Expiration timestamp. MUST NOT be null for AProof tokens

Time Bounds: Following least-authority principles, tokens SHOULD have the shortest practical validity window. A buffer of ± 60 seconds is RECOMMENDED for clock drift tolerance.

Intent Binding

Field	Type	Required	Description
<code>nonce</code>	String	Yes	Unique value preventing replay attacks. 12+ random bytes RECOMMENDED
<code>intent_hash</code>	String	Yes	SHA-256 hash of the canonicalized intent this token authorizes

The `intent_hash` cryptographically binds this token to a specific, evaluated intent. The hash is computed as:

```
intent_hash = "sha256:" + hex(SHA256(canonicalize(intent)))
```

Where `canonicalize()` follows RFC 8785 (JCS) for deterministic JSON serialization.

Capability Specification

Field	Type	Required	Description
cap	Object	Yes	The capability this token grants
cap.action	String	Yes	The action authorized (e.g., send, read, transfer)
cap.resource	URI	Yes	The resource this capability applies to
cap.constraints	Object	No	Additional constraints on the capability

Example Capability:

```
{
  "cap": {
    "action": "send",
    "resource": "mailto:user@example.com",
    "constraints": {
      "max_recipients": 10,
      "allowed_domains": ["example.com", "trusted.org"]
    }
  }
}
```

Proof Requirements

Field	Type	Required	Description
proof_level	Enum	Yes	Required cryptographic proof level: NONE, ATTESTATION, ZK
obligations	Array	No	List of obligations that must be satisfied before execution

Proof Levels:

- **NONE:** No additional proof required (low-risk actions)
- **ATTESTATION:** Signed attestation from trusted party required
- **ZK:** Zero-knowledge proof required (high-risk actions)

Delegation

Field	Type	Required	Description
<code>delegation_depth</code>	Integer	Yes	Number of delegations from root. 0 = issued by resource owner
<code>prf</code>	Array	No	CIDs of proof tokens in delegation chain

Delegation allows a token holder to create sub-tokens with equal or lesser authority. Each delegation increments `delegation_depth` and adds the parent token's CID to `prf`.

Signature Generation

The signature is computed over the concatenation of the Base64URL-encoded header and payload:^{[3][2]}

```
signature_input = Base64URL(header) + "." + Base64URL(payload)
signature = Sign(private_key, signature_input)
token = signature_input + "." + Base64URL(signature)
```

Supported Algorithms:

Algorithm	Curve	Key Type	DID Prefix
EdDSA	Ed25519	OKP	<code>did:key:z6Mk...</code>
ES256	P-256	EC	<code>did:key:zDna...</code>
ES256K	secp256k1	EC	<code>did:key:zQ3s...</code>

Signature Verification

Verifiers MUST perform the following checks in order:^{[4][2][3]}

Step 1: Structural Validation

1. Split token on "." → [header_b64, payload_b64, signature_b64]
2. Verify exactly 3 parts exist
3. Decode each part from Base64URL
4. Parse header and payload as JSON
5. Verify header.typ == "LPCT"

Step 2: Temporal Validation

1. Get current_time (Unix seconds, UTC)
2. Verify payload.nbf ≤ current_time (if nbf present)
3. Verify current_time < payload.exp
4. Allow ±60 second tolerance for clock drift

Step 3: Cryptographic Verification

1. Extract algorithm from header.alg
2. Extract public key from header.kid (DID resolution)
3. Reconstruct signature_input = header_b64 + "." + payload_b64
4. Verify signature using algorithm and public key
5. If verification fails, REJECT token

Step 4: Intent Binding Verification

1. Obtain the intent being executed
2. Compute expected_hash = "sha256:" + hex(SHA256(canonicalize(intent)))
3. Verify payload.intent_hash == expected_hash
4. If mismatch, REJECT token

Step 5: Delegation Chain Verification (if prf present)

1. For each proof_cid in payload.prf:
 - a. Resolve proof_cid to parent token
 - b. Verify parent token (recursively)
 - c. Verify parent.sub == current.iss (chain continuity)
 - d. Verify current.cap $\subseteq$ parent.cap (attenuation)
 - e. Verify current.exp <= parent.exp (time attenuation)
2. Verify delegation_depth == len(prf)

Step 6: Audience Verification

1. Verify payload.aud matches this adapter/resource
 2. If mismatch, REJECT token
-

Verification Pseudocode

```
def verify_token(token: str, intent: Intent, adapter_id: str) -> Result:
    # Step 1: Structural
    parts = token.split(".")
    if len(parts) != 3:
        return Error("Invalid token structure")

    header = json_decode(base64url_decode(parts[0]))
    payload = json_decode(base64url_decode(parts[1]))
    signature = base64url_decode(parts[2])

    if header.get("typ") != "LPCT":
        return Error("Invalid token type")

    # Step 2: Temporal
    now = current_unix_time()
    if payload.get("nbft", 0) > now + 60:
        return Error("Token not yet valid")
    if payload["exp"] < now - 60:
        return Error("Token expired")

    # Step 3: Cryptographic
    public_key = resolve_did_key(header["kid"])
```



```

signature_input = parts[^0] + "." + parts[^1]

if not verify_signature(header["alg"], public_key, signature_input, signature):
    return Error("Invalid signature")

# Step 4: Intent Binding
expected_hash = "sha256:" + sha256_hex(canonicalize(intent))
if payload["intent_hash"] != expected_hash:
    return Error("Intent hash mismatch")

# Step 5: Delegation Chain
for proof_cid in payload.get("prf", []):
    parent = resolve_and_verify(proof_cid)
    if parent.payload["sub"] != payload["iss"]:
        return Error("Delegation chain broken")
    if not is_attenuation(payload["cap"], parent.payload["cap"]):
        return Error("Capability escalation detected")

# Step 6: Audience
if payload["aud"] != adapter_id:
    return Error("Audience mismatch")

return Ok(payload)

```

Replay Prevention

Every Λ Proof token MUST have a unique content identifier (CID). Executors MUST maintain a store of seen token CIDs and reject duplicates.

Implementation:

```
token_cid = CIDv1(multicodec=dag-cbor, hash=sha256, content=token_bytes)
```

Tokens SHOULD be evicted from the replay store after their exp timestamp passes.

Token Revocation

Tokens MAY be revoked before expiration by the issuer or any issuer in the delegation chain. Revocation is performed by publishing the token's CID to a revocation registry.

Verifiers SHOULD check revocation status before accepting tokens. Revocation is permanent and affects all downstream delegations.

Canonicalization

For deterministic hashing, all JSON structures MUST be canonicalized using RFC 8785 (JSON Canonicalization Scheme):

1. Sort all object keys lexicographically
2. Remove insignificant whitespace
3. Encode as UTF-8

Example:

```
// Input (non-canonical)
{  "b": 2, "a": 1  }

// Output (canonical)
{"a":1,"b":2}
```

Security Considerations

1. **Private Key Protection:** Private keys MUST NOT be transmitted or stored in extractable form. Hardware security modules (HSMs) are RECOMMENDED for production deployments.
2. **Minimum Validity Windows:** Tokens SHOULD expire as quickly as practical. For one-time actions, validity windows of 5-15 minutes are RECOMMENDED.
3. **Delegation Limits:** Implementations MAY enforce maximum `delegation_depth` to limit blast radius of compromised keys.

4. **Clock Synchronization:** All parties MUST synchronize clocks via NTP. The ± 60 second tolerance accommodates minor drift but does not protect against grossly misconfigured clocks.
 5. **Algorithm Agility:** While multiple algorithms are supported, Ed25519 is RECOMMENDED.
alg: none MUST NOT be supported.^[5]
-

Example Token

Decoded:

```
// Header
{
  "alg": "EdDSA",
  "typ": "LPCT",
  "kid": "did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLGpbnnEGta2doK"
}

// Payload
{
  "iss": "did:key:z6MkhaXgBZDvotDkL5257faiztiGiC2QtKLGpbnnEGta2doK",
  "sub": "did:key:z6MkiTBz1ymuepAQ4HEHYSF1H8quG5GLVVQR3djdX3mDooWp",
  "aud": "adapter:email",
  "iat": 1736365200,
  "exp": 1736368800,
  "nonce": "a1b2c3d4e5f6g7h8i9j0k1l2",
  "intent_hash":
"sha256:b94d27b9934d3e08a52e52d7da7dabfac484efe37a5380ee9088f7ace2efcde9",
  "cap": {
    "action": "send",
    "resource": "mailto:alice@example.com",
    "constraints": {
      "max_attachments": 5,
      "max_size_bytes": 10485760
    }
  },
  "proof_level": "ATTESTATION",
  "obligations": ["mfa_verified"],
```

```
"delegation_depth": 0,
"prf": []
}
```

Encoded:

```
eyJhbGciOiJIJZERTQSIIsInR5cCI6IkpXQQ1QIiLCJraWQiOiJkaWQ6a2V5Ono2TWtoYVhnQ1pEdm90RGtMNTI1N2ZhaXp0aUdpQzJRdEtMR3Bibm5FR3RhMmRvSyJ9.eyJpc3MiOiJkaWQ6a2V5Ono2TWtoYVhnQ1pEdm90RGtMNTI1N2ZhaXp0aUdpQzJRdEtMR3Bibm5FR3RhMmRvSyIsInN1YiI6ImRpZDprZXk6ejZNa2lUQnoxeW11ZXBBUTRIRUhZU0YxSDhxdUc1R0xwV1FSM2RqZFgzburvb1dwIiwiaXVkiOiJpYWRhcHRlcjplbWFPbCIsIm1hdCI6MTczNjM2NTIwMCwiZXhwIjoxNzY4MDAwLCJub25jZSI6ImExYjJjM2Q0ZTVmNmM3aDhpOWowazFsMiIsIm1udGVudF9oYXNoIjoic2hhMjU2OmI5NGQyN2I5OTM0ZDNlMDhhNTJlNTJkN2RhN2RhYmZhYzQ4NGVmZTM3YTUzODBlZTkwdDhmN2FjZTJlZmNkZTk1LCJyXAiOnsiYWN0aW9uIjoic2VuZCIsInJlc291cmNlIjoibWFPbHRvOmFsaWNLQGV4YW1wbGUuY29tIn0sInByb29mX2xldmVsIjoiaVRURVNUQVRJT04iLCJvYmxpZ2F0aW9ucyI6WyJtZmFdmVyaWZpZWQ1XSwiZGVsZWhhdG1vb19kZXBoaCI6MCwicHJmIjpjbXX0.SIGNATURE_HERE
```

References

- RFC 7519: JSON Web Token (JWT)
- RFC 8785: JSON Canonicalization Scheme (JCS)
- UCAN Specification v1.0.0-rc.1
- W3C DID Core Specification
- did:key Method Specification

Document Status

Type: Defensive Publication / Public Specification

Version: 1.0.0

Date: January 8, 2026

License: CC0 (Public Domain)

Want me to refine any section, add more detail on delegation semantics, or adjust the technical depth?

**

-
1. <https://github.com/ucan-wg/spec>
 2. Ξ Constitution.pdf
 3. <https://jwt.io/introduction>
 4. <https://idura.eu/blog/jwt-validation-guide>
 5. <https://stytch.com/blog/rfc-7519-jwt-part-1/>
 6. $2.\Lambda$ RootContract II & III - MTPI - NODE.js.pdf
 - a. Λ proof Monorepo — Plain-language Overview V0.pdf
 - b. Λ -trace Overview.pdf
 - c. Λ proof Banking Protocols.pdf
 - d. Λ proof Healthcare Protocols V0.1.pdf
 - e. Λ dapps On Web4 - Developer Overview.pdf
 7. Building a Λ Dapp_ A Developer's Implementation Guide.pdf
 8. MTPI_Instructions.pdf
 9. MTPI RootContract.pdf
 10. MTPI_RootContract_Project.pdf
 11. Patent Search Results.pdf
 12. The Λ Proof Architecture_ A Comprehensive Analysis of its Implications for Web4.pdf
 13. The Λ Proof Architecture_ Web4.pdf
 14. Verifiable, Privacy-Preserving Applications.pdf

15. Λ Proof.pdf
16. <https://curity.io/resources/learn/jwt-best-practices/>
17. <https://supertokens.com/blog/what-is-jwt>
18. <https://instasafe.com/blog/the-different-token-types-and-formats-explained/>
19. <https://classic-app.nft.storage/docs/how-to/ucan/>
20. <https://www.loginradius.com/blog/engineering/guide-to-jwt>
21. <https://www.cloudatomiclab.com/crypto-capabilities/>
22. <https://www.vaadata.com/blog/jwt-json-web-token-vulnerabilities-common-attacks-and-security-best-practices/>
23. <https://docs.oasis-open.org/pkcs11/pkcs11-base/v2.40/os/pkcs11-base-v2.40-os.html>
24. <https://www.youtube.com/watch?v=Muhfrqw9gQA>
25. <https://docs.digicert.com/en/software-trust-manager/code-signing/sign-with-third-party-signing-tools/javascript-and-web-tokens/sign-json-web-tokens-with-jwt-io.html>
26. <https://developer.apple.com/documentation/cryptokit/authenticating-users-with-a-cryptographic-token>
27. Milestone-Vision-Timeframe.csv
28. Lets further develop the SAFE_ awesome—let's turn.pdf
29. Λ^P -Archivum.pdf
 - f. Λ proof $\times$ Mtpi Lawful-by-design Integration — Implementation Plan.pdf